

Registry Integrato di OpenShift

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere i componenti del registry integrato di OpenShift e il ruolo dell'Operator `image-registry`
- Configurare lo stato di gestione del registry (`Managed`, `Removed`, `Unmanaged`)
- Configurare il backend di storage del registry (PVC, Azure Blob Storage, emptyDir)
- Accedere al registry dall'interno e dall'esterno del cluster tramite Route
- Eseguire operazioni di push/pull/mirror di immagini con `podman`, `skopeo` e `oc image mirror`
- Abilitare e configurare il registry integrato su ARO, disabilitato di default

Teoria e Concetti Chiave

Cos'è il Registry Integrato di OpenShift

OpenShift include un **registry interno di immagini container** gestito dall'Operator `image-registry`. È il componente che:

- Riceve il push delle immagini prodotte dai Build (BuildConfig / Pipeline)
- Serve le immagini ai nodi del cluster durante il pull dei Pod
- Integra l'autenticazione con gli stessi token OAuth di OpenShift

Il registry integrato è accessibile all'interno del cluster tramite il Service DNS:

```
image-registry.openshift-image-registry.svc:5000
```

Può essere esposto esternamente tramite una Route nel namespace `openshift-image-registry`.

Architettura dell'Operator `image-registry`

Componente	Namespace	Descrizione
<code>Config</code> (CR)	<code>openshift-image-registry</code>	Risorsa principale: <code>configs.imageregistry.operator.openshift.io/cluster</code>
Deployment <code>image-registry</code>	<code>openshift-image-registry</code>	Pod che esegue il registry (distribution/registry v2)
Service <code>image-registry</code>	<code>openshift-image-registry</code>	Endpoint interno del cluster (porta 5000)

Componente	Namespace	Descrizione
Route <code>default-route</code>	<code>openshift-image-registry</code>	Endpoint esterno HTTPS (opzionale)
Secret <code>image-registry-tls</code>	<code>openshift-image-registry</code>	Certificato TLS firmato dalla CA interna

Stato di Gestione del Registry

Il campo `spec.managementState` nella CR `Config` del registry controlla il ciclo di vita dell'Operator:

Valore	Comportamento	Quando usarlo
<code>Managed</code>	L'Operator gestisce e riconcilia il registry	Produzione, ambienti con build integrate
<code>Removed</code>	L'Operator rimuove il Deployment del registry	Ambienti che non usano build integrate (es. solo pull da ACR)
<code>Unmanaged</code>	L'Operator non modifica il registry	Manutenzione, configurazione manuale temporanea

⚠ Attenzione: Su ARO, il registry è in stato `Removed` di default perché ARO non configura automaticamente lo storage persistente. Prima di abilitarlo, è necessario configurare un Azure Storage Account come backend.

Backend di Storage del Registry

Il registry richiede un backend di storage persistente per conservare le immagini attraverso i riavvii:

Backend	Configurazione YAML	Uso consigliato
Azure Blob Storage	<code>spec.storage.azure</code>	ARO in produzione
PVC (ReadWriteMany)	<code>spec.storage.pvc</code>	OCP on-premise con storage condiviso
S3	<code>spec.storage.s3</code>	AWS / S3-compatible (MinIO)
emptyDir	<code>spec.storage.emptyDir: {}</code>	Solo sviluppo/test (dati persi al riavvio)

⚠ Attenzione: `emptyDir` non è persistente. Le immagini vengono perse quando il Pod del registry viene riavviato. Non usare in produzione.

Accesso al Registry

Dall'interno del cluster

I Pod possono fare push/pull usando il nome DNS interno:

```
image-registry.openshift-image-registry.svc:5000/<namespace>/<imagestream>:<tag>
```

I token di autenticazione sono gli stessi token OpenShift. Il ServiceAccount del Pod deve avere il ruolo `system:image-puller` nel namespace dell'immagine.

Dall'esterno del cluster

Abilitando `spec.defaultRoute: true`, viene creata una Route HTTPS:

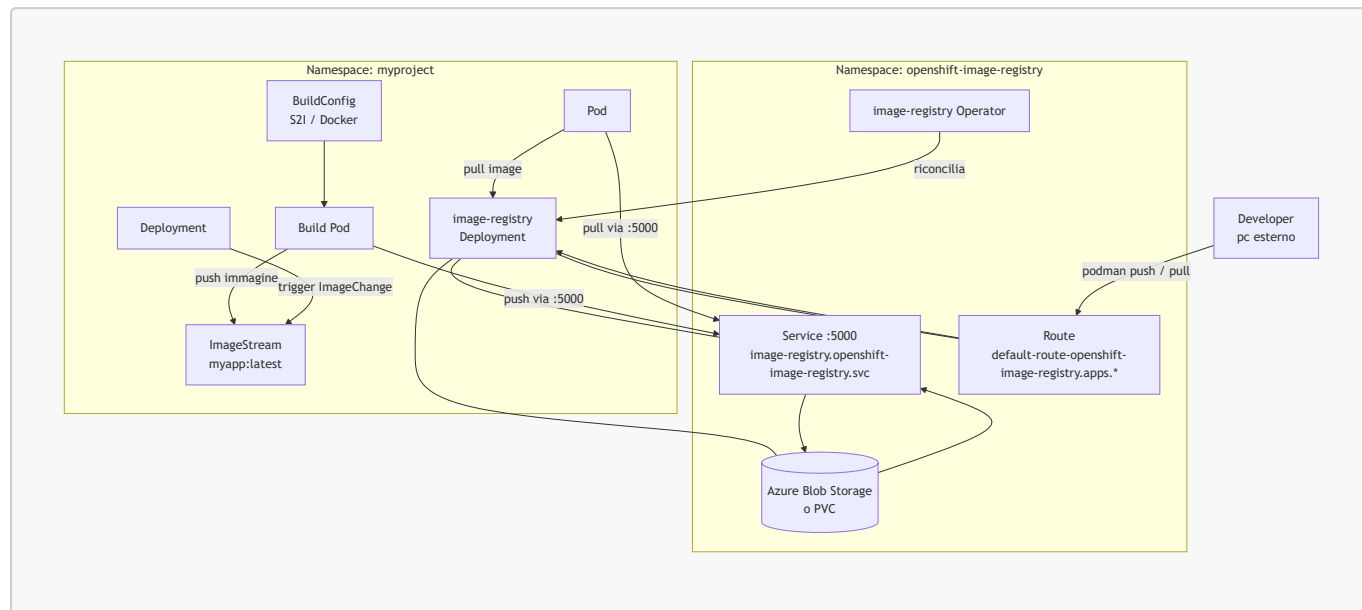
```
default-route-openshift-image-registry.apps.<cluster>.<region>.aroapp.io
```

Per fare login dall'esterno:

```
podman login -u $(oc whoami) -p $(oc whoami --show-token) \  
default-route-openshift-image-registry.apps.<cluster>.<region>.aroapp.io
```

Diagrammi

Componenti del Registry Integrato e Flussi Build → Push → Pull



Comandi Pratici con Output Atteso

```
# Verificare lo stato attuale del registry integrato
oc get configs.imageregistry.operator.openshift.io/cluster \
  -o jsonpath='{.spec.managementState}'

# Output atteso (ARO default):
# Removed
```

```
# Verificare i Pod del registry (se Managed)
oc get pods -n openshift-image-registry

# Output atteso:
# NAME                                READY   STATUS    RESTARTS   AGE
# image-registry-7d8b9f6c4-xkp2m      1/1     Running   0           2d
# node-ca-7cxlm                       1/1     Running   0           2d
# node-ca-8n9wt                       1/1     Running   0           2d
```

```
# Ottenere le informazioni sul registry interno
oc registry info

# Output atteso:
# image-registry.openshift-image-registry.svc:5000
```

```
# Ottenere l'endpoint esterno del registry (se defaultRoute abilitata)
oc registry info --public

# Output atteso:
# default-route-openshift-image-registry.apps.myaro.eastus.aroapp.io
```

```
# Login al registry interno dall'esterno usando il token OAuth
REGISTRY=$(oc registry info --public)
podman login -u $(oc whoami) -p $(oc whoami --show-token) ${REGISTRY}

# Output atteso:
# Login Succeeded!
```

```
# Push di un'immagine locale nel registry interno (es. dopo aver fatto podman
login)
REGISTRY=$(oc registry info --public)
```

```

NAMESPACE=myproject
podman tag myapp:1.0 ${REGISTRY}/${NAMESPACE}/myapp:1.0
podman push ${REGISTRY}/${NAMESPACE}/myapp:1.0

```

```

# Output atteso:
# Getting image source signatures
# Copying blob sha256:a1b2c3... done
# Copying config sha256:d4e5f6... done
# Writing manifest to image destination
# Storing signatures

```

```

# Copiare un'immagine da un registry esterno al registry interno con skopeo
skopeo copy \
  docker://registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  docker://image-registry.openshift-image-registry.svc:5000/myproject/ubi-
minimal:9.4 \
  --dest-creds $(oc whoami):$(oc whoami --show-token)

```

```

# Output atteso:
# Getting image source signatures
# Copying blob sha256:... done
# Copying config sha256:... done
# Writing manifest to image destination

```

```

# Verificare le immagini presenti in un ImageStream nel registry interno
oc get imagestream -n myproject

```

```

# Output atteso:
# NAME          IMAGE REPOSITORY
TAGS    UPDATED
# myapp          image-registry.openshift-image-registry.svc:5000/myproject/myapp
latest    2 minutes ago
# ubi-minimal    image-registry.openshift-image-registry.svc:5000/myproject/ubi-
minimal    9.4    5 minutes ago

```

```

# Abilitare la route esterna del registry (per accesso da fuori il cluster)
oc patch configs.imageregistry.operator.openshift.io/cluster \
  --type=merge \
  --patch='{ "spec": { "defaultRoute": true } }'

```

```

# Output atteso:
# configs.imageregistry.operator.openshift.io/cluster patched

```

```

# Mirror di un'immagine tra due namespace con oc image mirror
oc image mirror \

```

```
image-registry.openshift-image-registry.svc:5000/myproject/myapp:1.0 \  
image-registry.openshift-image-registry.svc:5000/staging/myapp:1.0 \  
-a /var/run/secrets/openshift.io/push/.dockercfg
```

```
# Output atteso:
```

```
# phase 0:
```

```
# image-registry.openshift-image-registry.svc:5000/myproject/myapp:1.0 ->  
staging/myapp:1.0
```

```
# info: Mirroring completed in 3.2s (12.5MB/s)
```

Esempi YAML Completi

Config del Registry Operator con Azure Blob Storage e Route Esterna

```
# Risorsa singleton del registry Operator di OpenShift
# Nome fisso: "cluster" – non modificare il metadata.name
apiVersion: imageregistry.operator.openshift.io/v1
kind: Config
metadata:
  name: cluster # ← nome fisso obbligatorio
spec:
  # Stato di gestione dell'Operator
  managementState: Managed # ← Managed | Removed | Unmanaged

  # Numero di repliche del registry (1 per dev, 2+ per alta disponibilità)
  replicas: 2

  # Abilita la Route esterna HTTPS per accesso da fuori del cluster
  defaultRoute: true # ← false = solo accesso interno

  # Configurazione del backend di storage
  storage:
    azure:
      # Nome del container Blob nell'Azure Storage Account
      container: openshift-registry # ← nome del blob container in Azure

      # Account Name dello Storage Account Azure
      # Viene letto dal Secret "image-registry-private-configuration" come env var
      # accountName è opzionale se il Secret contiene
      REGISTRY_STORAGE_AZURE_ACCOUNTNAME
      accountName: myarostorage # ← nome dello Storage Account Azure

  # Richieste e limiti di risorse per il Pod del registry
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 1Gi

  # Configurazione del proxy (se il cluster usa un proxy per uscire su Internet)
  proxy:
    http: "" # ← URL del proxy HTTP se necessario
    https: ""
    noProxy: ".cluster.local,.svc,localhost" # ← escludi traffico interno

  # Configurazione del logging
  logging: 2 # ← livello 0-10 (default: 2)

  # Configurazione della readinessProbe (override dell'Operator)
```



```
    readOnly: false                                # ← false = push abilitato
---
# Secret con le credenziali dello Storage Account Azure per il registry
# Deve essere nel namespace openshift-image-registry
apiVersion: v1
kind: Secret
metadata:
  name: image-registry-private-configuration-user  # ← nome specifico atteso
dall'Operator
  namespace: openshift-image-registry
type: Opaque
stringData:
  # Chiave di accesso allo Storage Account Azure (key1 o key2)
  REGISTRY_STORAGE_AZURE_ACCOUNTKEY: "base64-encoded-storage-account-key"
```

Note Specifiche per Azure ARO

Abilitazione del Registry su ARO

Su ARO, il registry integrato è in stato **Removed** di default perché **non viene configurato automaticamente** un backend di storage persistente. Per abilitarlo:

Passo 1: Creare uno Storage Account Azure

```
# Creare un Resource Group dedicato (opzionale, si può usare quello ARO)
az storage account create \
  --name myaroregistry \
  --resource-group myaro-rg \
  --location eastus \
  --sku Standard_LRS \
  --kind StorageV2 \
  --access-tier Hot

# Creare il container Blob
az storage container create \
  --name openshift-registry \
  --account-name myaroregistry

# Recuperare la chiave di accesso
STORAGE_KEY=$(az storage account keys list \
  --account-name myaroregistry \
  --resource-group myaro-rg \
  --query "[0].value" -o tsv)
```

Passo 2: Creare il Secret con le credenziali

```
oc create secret generic image-registry-private-configuration-user \
  --from-literal=REGISTRY_STORAGE_AZURE_ACCOUNTKEY="${STORAGE_KEY}" \
  -n openshift-image-registry
```

Passo 3: Abilitare il registry con il backend Azure

```
oc patch configs.imageregistry.operator.openshift.io/cluster \
--type=merge \
--patch='{
  "spec": {
    "managementState": "Managed",
    "storage": {
      "azure": {
        "container": "openshift-registry",
        "accountName": "myaroregistry"
      }
    },
    "replicas": 2,
    "defaultRoute": true
  }
}'
```

Pull Secret di Sistema su ARO

ARO utilizza un pull secret globale (`pull-secret` nel namespace `openshift-config`) per autenticare il pull delle immagini Red Hat da `registry.redhat.io`. Questo secret è pre-configurato da ARO e **non deve essere modificato direttamente** senza seguire la procedura corretta (aggiungere credenziali senza rimuovere quelle esistenti).

```
# Visualizzare il contenuto del pull secret globale (senza rivelare le
credenziali)
oc get secret pull-secret -n openshift-config \
-o jsonpath='{.data.\.dockerconfigjson}' | \
base64 -d | jq '.auths | keys'

# Output atteso:
# [
#   "arosvc.azurecr.io",
#   "registry.connect.redhat.com",
#   "registry.redhat.io"
# ]
```

Riepilogo dei Punti Chiave

- Il registry integrato di OpenShift è gestito dall'Operator `image-registry` tramite la CR singleton `configs.imageregistry.operator.openshift.io/cluster`
 - Lo stato `managementState` controlla il ciclo di vita: `Managed` (attivo), `Removed` (rimosso), `Unmanaged` (non riconciliato)
 - Su ARO il registry è `Removed` di default: occorre configurare prima un Azure Storage Account (Blob) come backend persistente
 - Il Service interno `image-registry.openshift-image-registry.svc:5000` è usato automaticamente dai build e dai Pod nel cluster
 - La Route esterna (`defaultRoute: true`) espone il registry all'esterno con TLS; l'autenticazione usa lo stesso token OAuth di `oc whoami --show-token`
 - `skopeo copy` e `oc image mirror` permettono di copiare immagini tra registry senza pull/push locale
 - Il pull secret globale `openshift-config/pull-secret` autentica i nodi verso `registry.redhat.io` e `arosvc.azurecr.io`
-

Domande di Verifica

1. Qual è il valore di **managementState** del registry integrato su un cluster ARO appena installato?

- A) **Managed**
- B) **Unmanaged**
- C) **Removed** ✓
- D) **Disabled**

2. Quale backend di storage è consigliato per il registry integrato su un cluster ARO in produzione?

- A) **emptyDir**
- B) **PVC con ReadWriteOnce**
- C) **Azure Blob Storage** ✓
- D) **NFS**

3. Come si ottiene l'endpoint del registry integrato per l'accesso dall'interno del cluster?

- A) **oc get route -n openshift-image-registry**
- B) **oc registry info** ✓
- C) **oc get service image-registry -n default**
- D) **oc describe configs.imageregistry.operator.openshift.io/cluster**

4. Quale comando permette di copiare un'immagine da **registry.access.redhat.com** al registry interno senza effettuare un pull/push locale?

- A) **oc image copy**
- B) **podman copy**
- C) **skopeo copy** ✓
- D) **oc tag**

5. Quale Secret deve essere creato nel namespace **openshift-image-registry** per fornire le credenziali dello Storage Account Azure al registry Operator?

- A) **image-registry-tls**
- B) **image-registry-private-configuration-user** ✓
- C) **azure-storage-credentials**
- D) **pull-secret**